Echo: Problems Documentation

Browser voice AI pipeline: Speech-to-Text, LLM, Text-to-Speech

Project links

Live app	https://echo-production-aa12.up.railway.app/
GitHub repo	https://github.com/Aryan-Rustagi/Echo.git

Overview

Echo is a browser voice assistant built from independent stages: microphone capture, speech-to-text (STT), an LLM, and text-to-speech (TTS). This document records the problems faced while building it. Each entry lists the problem, its root cause, the resolution, and the main lesson. Problems 1 to 4 are design and experience problems that shaped the architecture. Problems 5 to 8 are bugs found while building the speech pipeline.

#	Problem	Area	Fix in one line
1	Silent waiting with the REST (Whisper) approach	UX / Architecture	Switched from REST Whisper to WebSocket streaming
2	Dead silence while the LLM writes its full answer	Latency	Speak the first sentence while the LLM writes the rest
3	Third-party API failures breaking the app	Reliability	Automatic fallback to Whisper if a realtime provider fails
4	AI and user talking over each other	Conversation flow	Barge-in: stop audio and cancel the response on user speech
5	Sentence splitting bug (the "Dr." problem)	Bug / Text processing	Regex split: punctuation + space, minimum 15 characters
6	Audio playing out of order (race condition)	Bug / Concurrency	FIFO playback queue keeps sentence order
7	Whisper hallucinating on empty audio	Bug / STT	Skip the API if audio is under 4 KB
8	Ghost audio playing after clicking Stop	Bug / Cancellation	AbortController per request and clear the queue on Stop

Design and experience problems

Problem 1: Silent waiting with the REST (Whisper) approach

UX / Architecture

Problem	The first version recorded audio, and the user clicked Stop and waited. Nothing appeared on screen for 2 to 3 seconds, so the user could not tell whether the microphone had picked up their voice or what was being transcribed. It did not feel like a real conversation.
Root cause	A REST call is request/response. The audio is only uploaded after recording ends, so no text can exist until the whole file has been sent and transcribed. There is no feedback during speech.
Resolution	Moved from sending recorded audio files to real-time WebSocket streaming with Speechmatics and Deepgram. Audio streams continuously while the user speaks, and the words appear on screen word by word.
Lesson	Streaming moves the feedback from after the speech to during the speech. Perceived latency matters as much as actual latency.

Problem 2: Dead silence while the LLM writes its full answer

Latency

Problem	The response had to be fully written before it could be turned into speech. A three-sentence answer meant 3 to 4 seconds of silence before Echo said anything.
Root cause	Text-to-speech started only after the complete LLM reply arrived, so the stages ran one after another instead of overlapping.
Resolution	The LLM response is split sentence by sentence. As soon as the first sentence is complete it is spoken immediately, while the LLM is still writing the second sentence in the background. This cut the wait to roughly a second.
Lesson	Overlap the pipeline stages (pipelining). Time to first audio matters more than time to full answer.

Problem 3: Third-party API failures breaking the app

Reliability

Problem	Echo depends on external APIs. A key can run out of credits, a rate limit can be hit, or the connection can drop. Any of these could crash the app with an error screen.
Root cause	The pipeline had a single path per stage. When the realtime STT provider failed, there was nothing else to try.
Resolution	Added an automatic backup plan. If Speechmatics or Deepgram fails, the app switches to the Whisper REST route in the background, shows a small notice, and the user can keep talking.
Lesson	Design for failure. A fallback should be triggered by a specific provider failure and should always show the user what happened.

Problem 4: AI and user talking over each other

Conversation flow

Problem	In early tests, if the AI was speaking and the user started to talk, both voices played at the same time. A real person would stop talking when interrupted.
Root cause	Nothing was watching for user speech while audio was playing, and there was no way to cancel the response that was already in progress.
Resolution	Added an interruption (barge-in) feature. The moment the user starts speaking or clicks Stop, the app cuts off the AI audio, cancels the previous response, and immediately listens to what the user says next.
Lesson	Barge-in means detecting speech during playback, then stopping audio and aborting in-flight requests.

Bugs in the speech pipeline

Problem 5: Sentence splitting bug (the "Dr." problem)

Bug / Text processing

Problem	To start speaking quickly, the AI text is split at periods. A simple split on '.' broke titles like "Dr. Smith" and abbreviations like "e.g." into separate broken sentences, so the voice said "Dr", paused, then said "Smith".
Root cause	A period does not always end a sentence. Abbreviations and titles contain periods too, and a plain split cannot tell the difference.
Resolution	Wrote a regex rule that only splits on sentence-ending punctuation followed by a space, and only when the sentence is at least 15 characters long. This stops titles and abbreviations from being split.
Lesson	Naive string splitting fails on real language. Add context rules such as punctuation followed by whitespace plus a minimum length.

Problem 6: Audio playing out of order (race condition)

Bug / Concurrency

Problem	Sentence 1 and Sentence 2 are sent to the Text-to-Speech API at the same time to save time. Sentence 2 was very short (for example "Yes, absolutely!"), so its audio finished converting before Sentence 1. The browser started playing Sentence 2 first.
Root cause	Parallel requests finish in whatever order they complete, not the order they were sent. The playback code played audio as soon as it arrived.
Resolution	Built a FIFO queue (a playlist array). Even if Sentence 2 finishes downloading first, it is held in memory and only starts after Sentence 1 has finished playing.
Lesson	Parallel fetching is fine, but playback must be sequential. Keep the order in the queue, not in the network timing.

Problem 7: Whisper hallucinating on empty audio

Bug / STT

Problem	When testing the fallback recorder, if the user clicked Start and then Stop without saying anything, the audio file was silent. The Whisper API still guessed and returned text such as "Thank you for watching!" or "Thanks."
Root cause	Whisper was trained on large amounts of video subtitles, where silence often maps to sign-off phrases. Given an empty or near-empty file, it fills the gap with those phrases.
Resolution	Added a file-size check in the backend. If the recorded audio is smaller than 4 KB (4,000 bytes in the code), the file is just an empty header with no voice. The backend skips the API completely and returns an empty transcript.
Lesson	Do not send empty input to a model. Validate cheaply before paying for an API call.

Problem 8: Ghost audio playing after clicking Stop

Bug / Cancellation

Problem	While the AI was speaking, clicking Stop Recording or interrupting stopped the first sentence. But Sentence 2 was still downloading in the background, and a second later it started playing on its own.
Root cause	Stopping the current audio did not cancel the pending network requests or empty the playback queue. The late-arriving audio was still added to the queue and played.
Resolution	Attached an AbortController to every sentence request. When the user clicks Stop or starts speaking, all pending downloads are aborted and the queue is cleared, so no leftover audio plays.
Lesson	Cancel the whole chain, not just the visible part. In-flight requests need an AbortController, and queues need to be cleared.